

プロジェクトの種類と参照関係図

先に結論

テスト

最初は **最小構成で4本** がいちばん良いです。

1. PdfUtility.Core
2. PdfUtility.Barcode
3. PdfUtility.SampleApp
4. PdfUtility.Tests

そのあと、必要になったら

5. PdfUtility.PreviewWeb

を足します。

いきなり Web まで作るより、まず DLL と動作確認アプリを先に作る方が、止まりにくいです。

1. 最初に作るプロジェクトの種類

A. PdfUtility.Core

プロジェクト種類

Class Library

おすすめ

言語: VB.NET でも C# でも可
ターゲット: .NET Standard 2.0

理由

ここが本体DLLです。

PDF読込、文字描画、図形描画、保存、結合、分割などを持たせます。

あなたは VB.NET に慣れているので、**Core は VB.NET で始めても全然あり**です。

B. PdfUtility.Barcode

プロジェクト種類

Class Library

おすすめ

言語: VB.NET でも可

ターゲット: .NET Standard 2.0

理由

DotNetBarcode をラップする層です。

IBarcodeGenerator の実装をここに置きます。

本体DLLに直書きせず、**バーコードだけ分離**しておくと後で差し替えやすいです。

C. PdfUtility.SampleApp

プロジェクト種類

最初はこれがおすすめです。

Console App

理由

最初の確認は GUI よりコンソールの方が早いです。
標準テンプレートとしてコンソール アプリも用意されています。

役割

DLLの動作確認
テキスト描画確認
矩形描画確認
QR / CODE39 貼付確認
結合 / 分割確認

補足

もし最初から簡単な画面が欲しければ、後で

Windows Forms App

を追加してもいいですが、最初の1本は Console で十分です。

D. PdfUtility.Tests

プロジェクト種類

Unit Test Project

おすすめ

MSTest か xUnit のどちらでもよい
あなたなら最初は **MSTest** で十分

.NET の標準テンプレートには単体テスト プロジェクトもあります。

役割

PDF読込テスト

テキスト描画テスト
図形描画テスト
バーコード生成テスト
保存テスト

E. PdfUtility.PreviewWeb

プロジェクト種類

ASP.NET Core Web App

おすすめ

言語: **C# 推奨**
実装スタイル: **Razor Pages 寄り**

理由

プレビュー画面は Web 側の担当にした方がきれいです。
ASP.NET Core アプリのテンプレートは標準で用意されています。

役割

PDF背景表示
テキスト / QR の配置確認
ドラッグで位置調整
座標保存
実PDF出力呼び出し

2. あなた向けのおすすめ順

私ならこの順で作ります。

第1段階

PdfUtility.Core
PdfUtility.Barcode
PdfUtility.SampleApp
PdfUtility.Tests

第2段階

PdfUtility.PreviewWeb

この順番だと、まず DLL が完成してから UI を作る 流れになるので、かなり安定します。

3. Visual Studio での具体的な作り方イメー

最初に作るソリューション

Blank Solution でもいいですし、最初に Class Library を作ってそこから増やしても大丈夫です。

私ならこうします。

手順

1. 新しいソリューション作成
2. **Class Library** で PdfUtility.Core
3. **Class Library** で PdfUtility.Barcode
4. **Console App** で PdfUtility.SampleApp
5. **Unit Test Project** で PdfUtility.Tests
6. 後から **ASP.NET Core Web App** で PdfUtility.PreviewWeb

4. 参照関係図

ここがかなり大事です。
依存を逆流させない方が、後で楽です。

まず結論の図

PdfUtility.Core



PdfUtility.Barcode

PdfUtility.Core ← PdfUtility.SampleApp

PdfUtility.Barcode ← PdfUtility.SampleApp

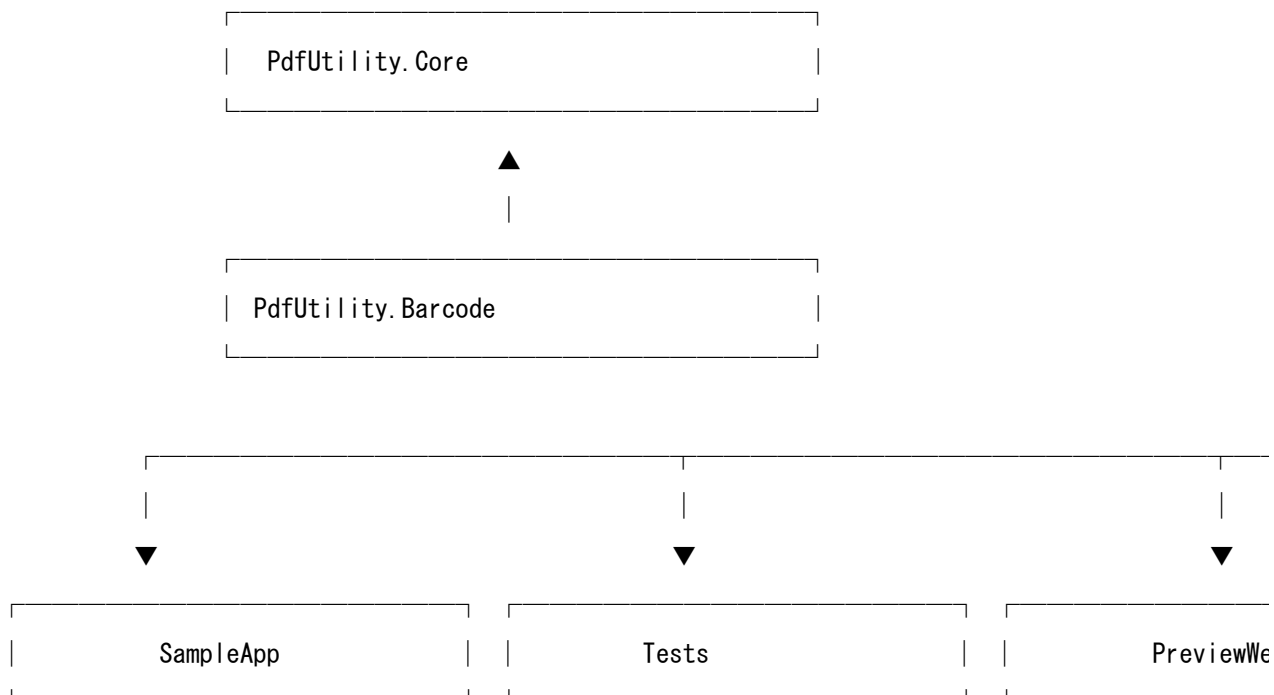
PdfUtility.Core ← PdfUtility.Tests

PdfUtility.Barcode ← PdfUtility.Tests

PdfUtility.Core ← PdfUtility.PreviewWeb

PdfUtility.Barcode ← PdfUtility.PreviewWeb

もう少し見やすく書くとこうです。



5. それぞれの参照ルール

PdfUtility.Core

参照するもの

できるだけ 他プロジェクトを参照しない

理由

ここが一番土台です。
他に依存しない方が使い回しやすいです。

PdfUtility.Barcode

参照するもの

PdfUtility.Core

理由

IBarcodeGenerator や BarcodeGenerateOptions を Core に置くなら、
Barcode 側がそれを実装する形になるからです。

PdfUtility.SampleApp

参照するもの

PdfUtility.Core
PdfUtility.Barcode

理由

本体DLLとバーコード実装の両方を使って、実際にPDFを出力するためです。

PdfUtility.Tests

参照するもの

PdfUtility.Core

PdfUtility.Barcode

理由

本体処理とバーコード処理をテストするためです。

ASP.NET Core のドキュメントでも、**単体テストと結合テストは別プロジェクトに分ける**のが推奨されています。

PdfUtility.PreviewWeb

参照するもの

PdfUtility.Core

PdfUtility.Barcode

理由

Web画面から配置情報を受け取って、最終的に DLL を呼ぶためです。

6. 依存関係でやってはいけないこと

これはかなり大事です。

NG例

PdfUtility.Core → PdfUtility.PreviewWeb を参照する

PdfUtility.Core → PdfUtility.SampleApp を参照する

PdfUtility.Barcode → PdfUtility.SampleApp を参照する

理由

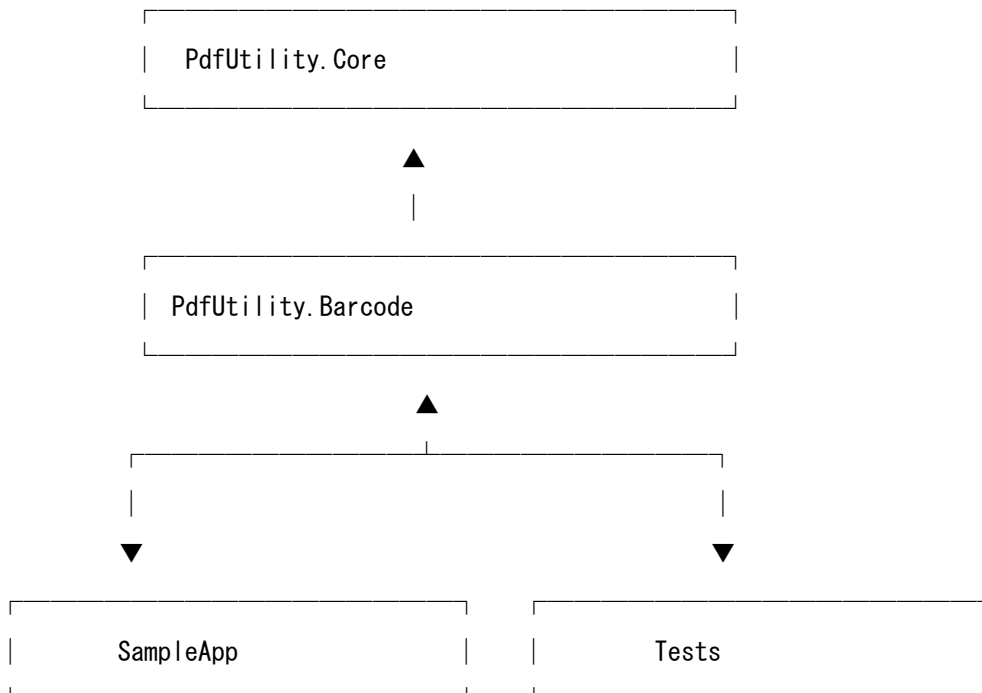
DLL 本体が UI に依存すると、再利用しにくくなります。

Core は下層、SampleApp / PreviewWeb は上層

この形を守るのが大事です。

7. 最初の最小構成版の参照関係図

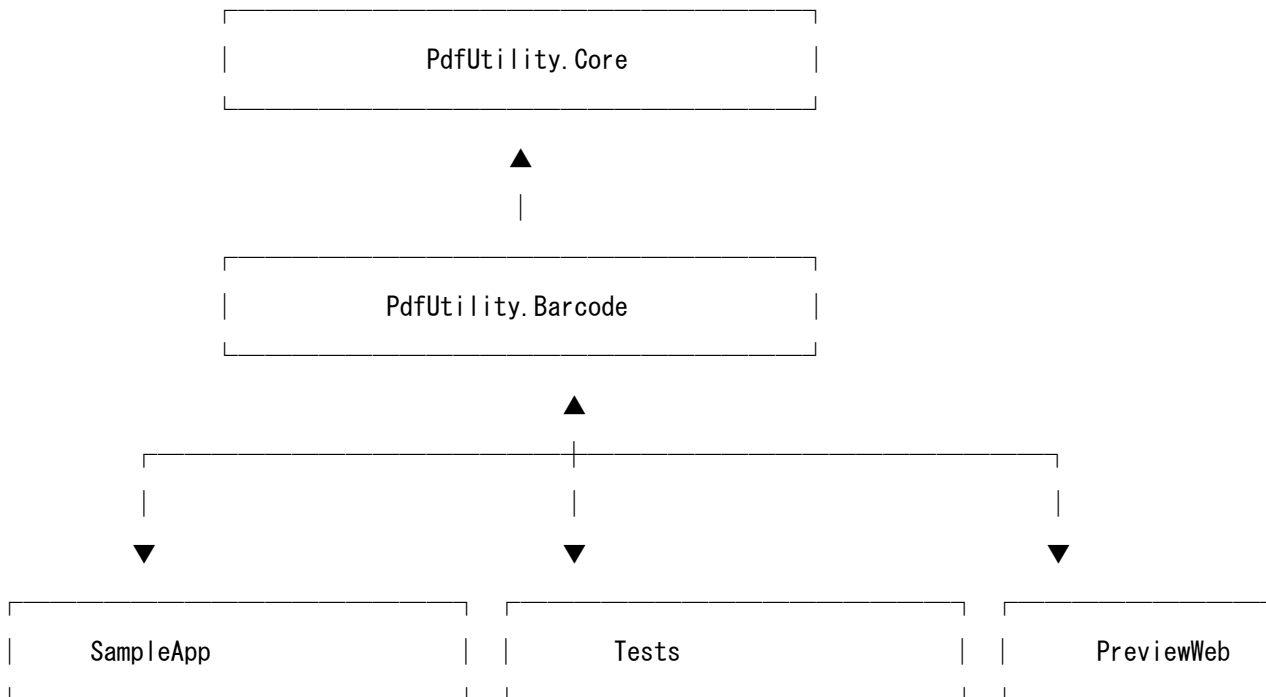
Web をまだ作らないなら、最初はこれだけで十分です。



これが最初の完成形としてちょうどいいです。

8. 将来の拡張版の参照関係図

プレビューWebを足したあとはこうです。



9. あなた向けのおすすめ言語構成

実務的にはこうするのがかなり自然です。

案A

Core: **VB.NET**
Barcode: **VB.NET**
SampleApp: **VB.NET**
Tests: **VB.NET**
PreviewWeb: **C#**

これはかなりおすすめです。

DLLは慣れたVB.NETで進める、
Webは作りやすいC#で作る、
という分担です。

案B

全部 C#

これもありですが、今のあなたなら最初は無理に全部C#に寄せなくていいです。

10. 私のおすすめ最終形

今のあなたに一番合うのはこれです。

最初に作るプロジェクト

PdfUtility.Core → Class Library

PdfUtility.Barcode → Class Library

PdfUtility.SampleApp → Console App

PdfUtility.Tests → Unit Test Project

後で追加

PdfUtility.PreviewWeb → ASP.NET Core Web App

参照関係

PdfUtility.Barcode → PdfUtility.Core

PdfUtility.SampleApp → PdfUtility.Core, PdfUtility.Barcode

PdfUtility.Tests → PdfUtility.Core, PdfUtility.Barcode

PdfUtility.PreviewWeb → PdfUtility.Core, PdfUtility.Barcode

11. ひとこと言おうと

「Core を中心にして、Barcode はその外側、SampleApp / Tests / PreviewWeb はさらに外側
この同心円みたいな構成が、一番きれいです。

これなら途中で止まりにくいですし、

奨られています。まずは1つのテストプロジェクトで始めて、後で `UnitTests` と `IntegrationTests` にう

うけてもいいです。